

Image and Video Generations

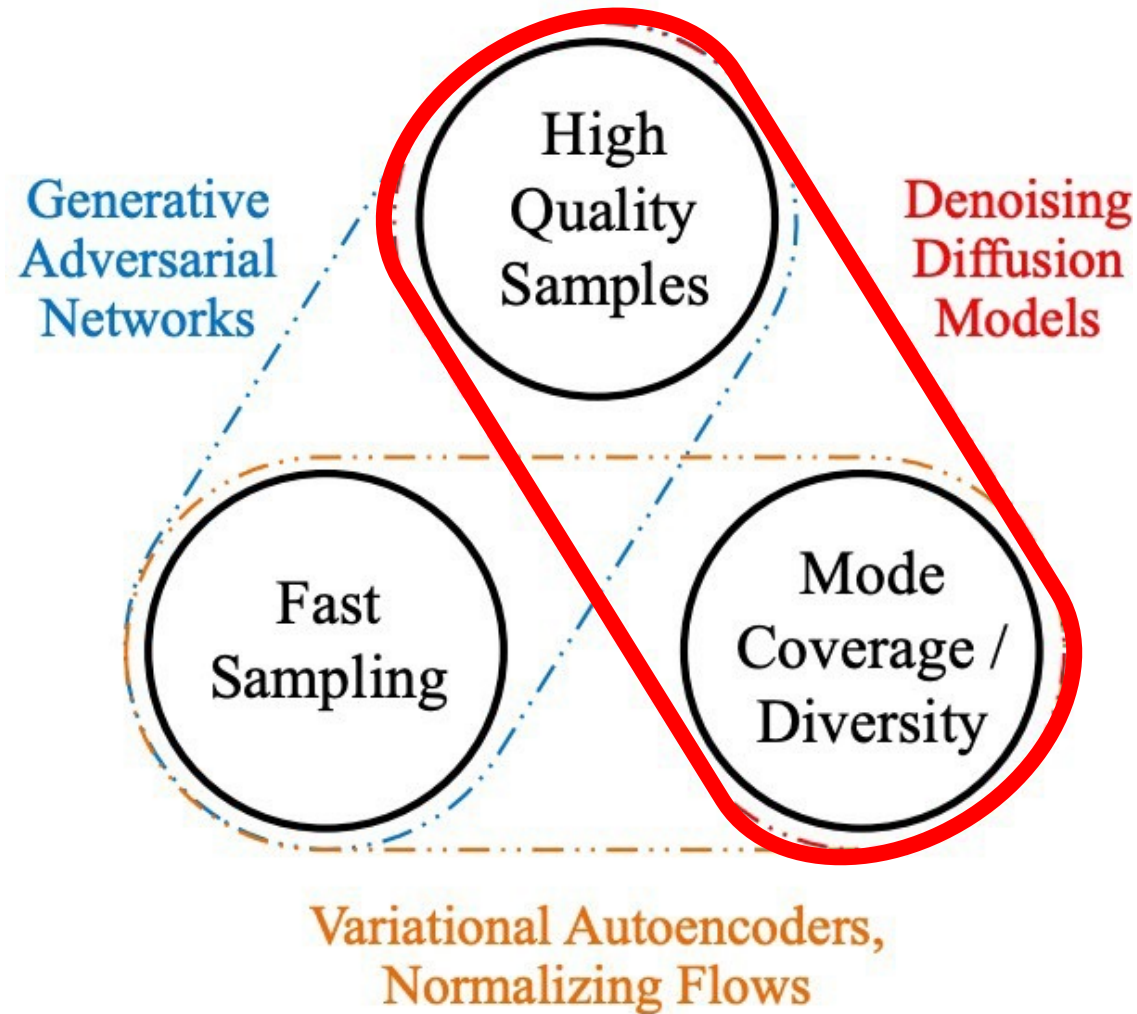
Lecture 9: Probability Flow ODE / DPE Solver

劉育綸

Yu-Lun (Alex) Liu



Generative Models – Comparison



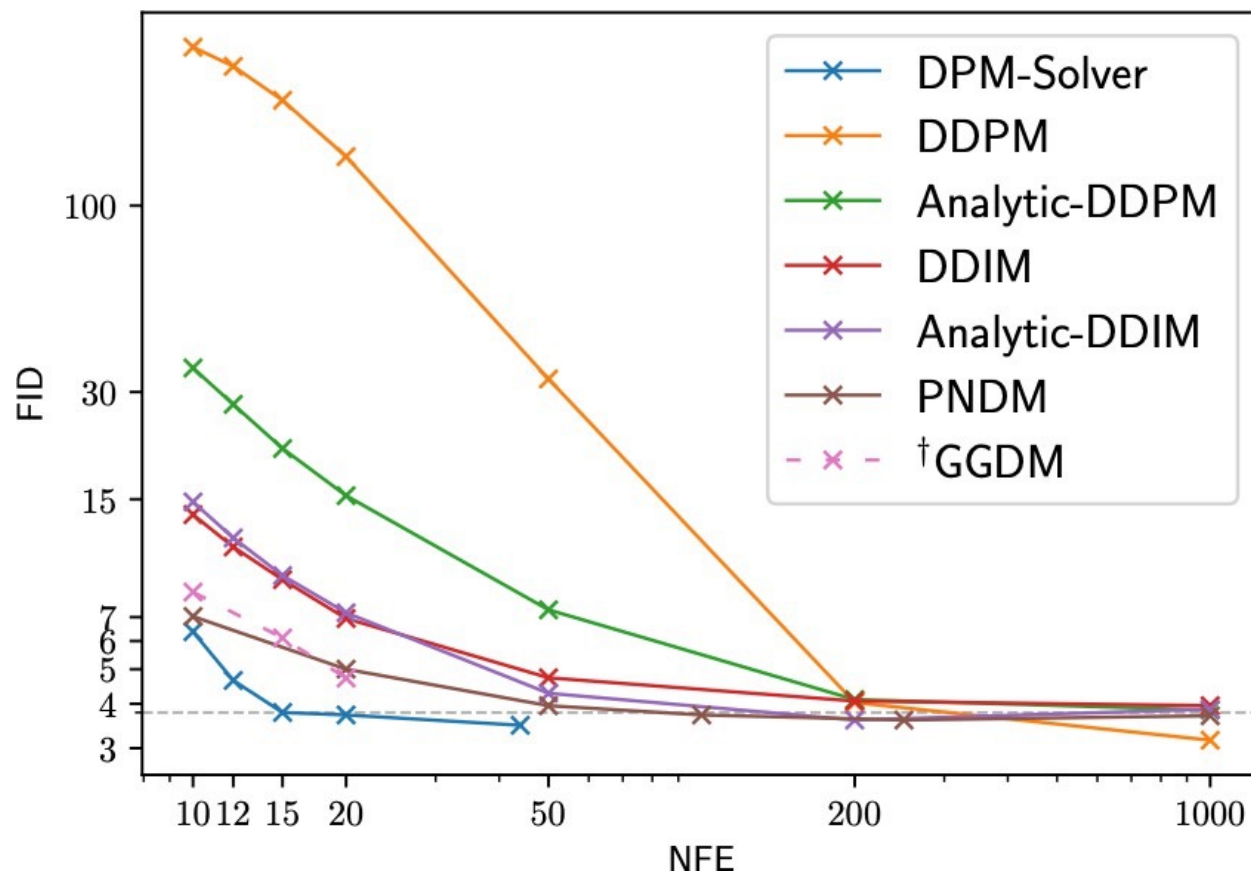
Generation Speed

To generate data in good quality,

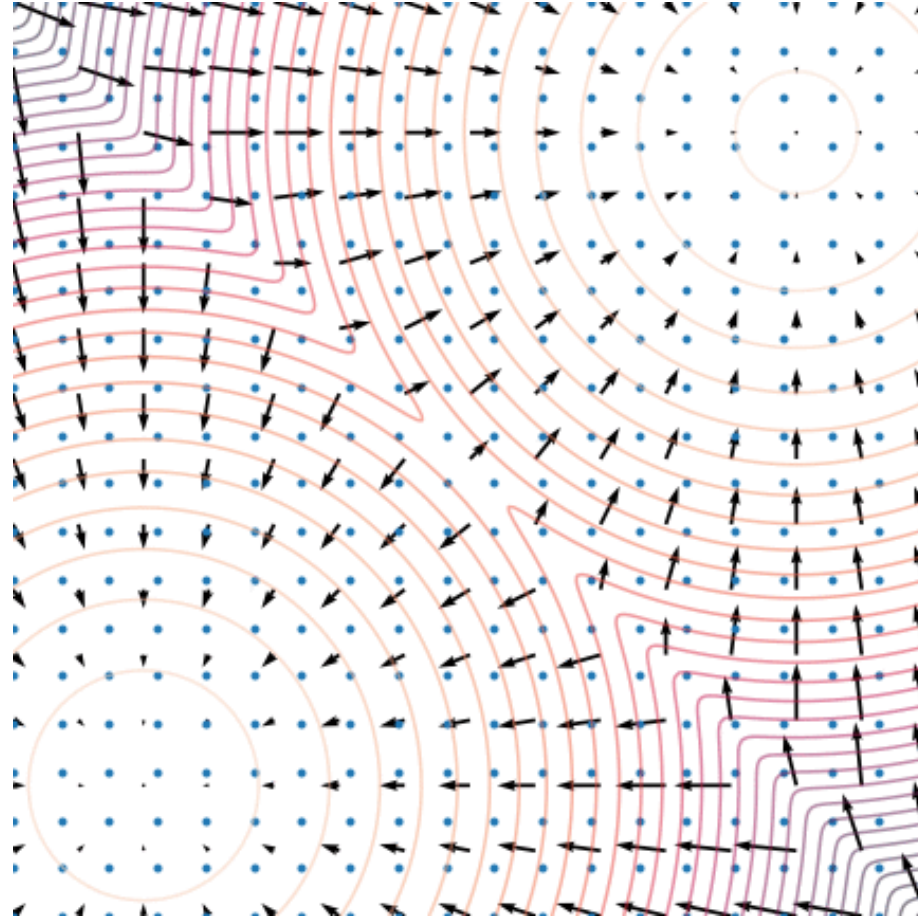
- **DDPM** requires 200-1000 denoising steps.
- **DDIM** requires 50-200 denoising steps.

Faster Generation

How can we generate good quality data in < 10 steps?



Langevin Dynamics



Stochastic Differential Equations

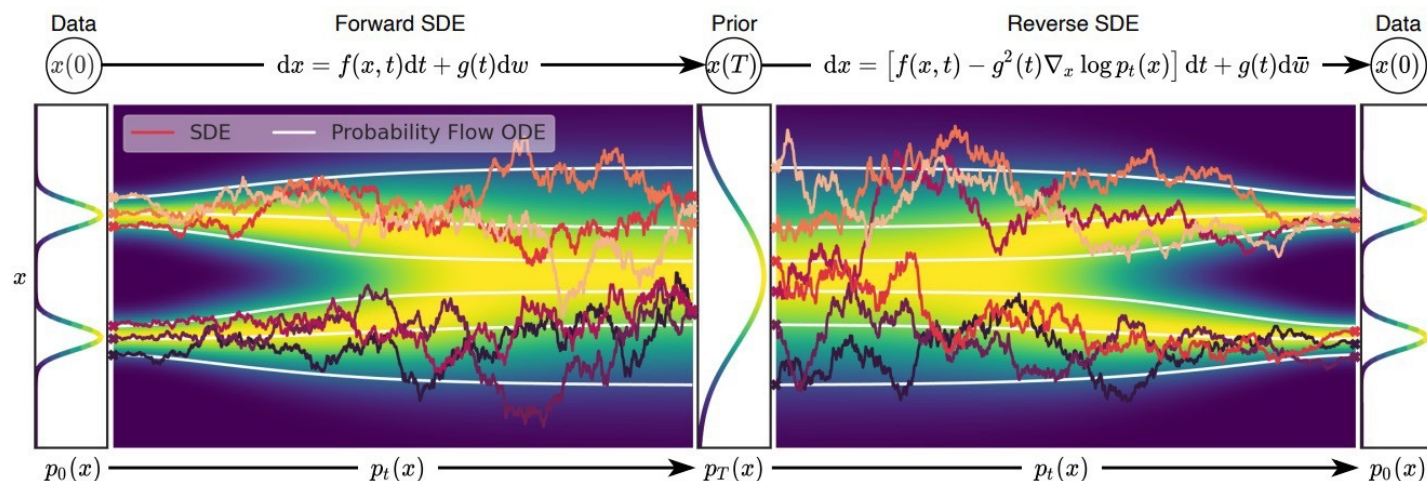
In a **continuous** time domain, the **forward** process is described by the following **stochastic differential equation (SDE)**:

這個式子其實跟 DDPM/DDIM 的正向加噪一樣，只是現在在 continuous time domain，所以有 $x(t)$ 而非 $\mathbf{x}_t$

$$dx(t) = f(t)x(t)dt + g(t)d\mathbf{w}, \quad x(0) \sim q(\mathbf{x}_0)$$

Drift term Diffusion term

- $f(t)$: Drift coefficient
- $g(t)$: Diffusion coefficient
- $d\mathbf{w}$: Infinitesimal white noise (called Brownian motion)
- Drift term: deterministic 的方向
類似於 DDPM 中的 $\sqrt{1 - \beta_t}$ scale
- Diffusion term: 隨機擾動，類似於
DDPM 中加入的 noise $\sqrt{\beta_t}\epsilon$



每個 **sample** 的軌跡都是一個完整的 $x(t)$ ：
從真實資料分布 $x(0)$ 出發，沿著某條隨機路徑，最終到達 **noise** 分佈 $x(T)$

DDPM vs. SDE

DDPM (Discrete)

- $\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\varepsilon}$
- Time step $t = 0, 1, \dots, T$
- $\mathbf{x}_t = \sqrt{1 - \beta_t} \mathbf{x}_{t-1} + \sqrt{\beta_t} \boldsymbol{\varepsilon}$

SDE (Continuous)

- $x(t)$ = 某個 SDE 的解
- Continuous time $t \in [0, T]$
- $dx(t) = f(t)x(t)dt + g(t)d\mathbf{w}$

Q. 為什麼要用 continuous time SDE?

Every diffusion process has some corresponding stochastic differential equation in the continuous time domain.

所有離散的 diffusion models 都可以看作 SDE 的離散化版本

DDPM, DDIM, Score-based models 都是 SDE 的特例

Stochastic Differential Equations

Its **reverse** process is also formulated as another **stochastic differential equation (SDE)**:

反向去噪過程也有對應的 SDE

$$dx(t) = \left[f(t)x(t)dt - g^2(t) \underbrace{\nabla_{x(t)} \log p_t(x(t))}_{\text{Score}} \right] dt + g(t)dw$$

Score 與 noise
只差了一個 scale

$$= \left[f(t)x(t) + \frac{g^2(t)}{\sqrt{1 - \bar{\alpha}_t}} \hat{\epsilon}_{\theta}(\underbrace{x(t)}_{x_t}, t) \right] dt + g(t)dw$$

$$x(T) \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

Forward Process

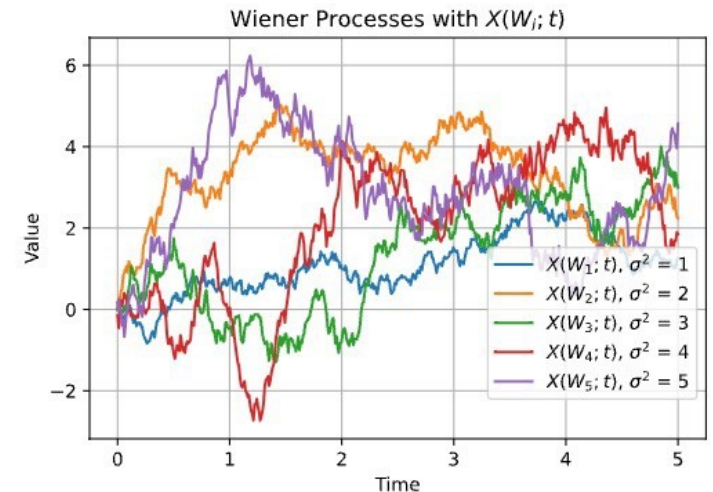
$$dx(t) = f(t)x(t)dt + g(t)d\mathbf{w}$$

- What is $d\mathbf{w}$? 這一項加入隨機擾動，使得整個過程變成 SDE
沒有這項的話，整個過程就只是一個 ODE，會好解很多
- What are $f(t)$ and $g(t)$?

Wiener Process (Brownian Motion)

A real-valued continuous-time stochastic process characterized by the following properties:

1. Initial condition: $\mathbf{w}(0) = 0$ almost surely.
Probability is 1.
2. Normally distributed increments:
For every $0 \leq s < t$, $\mathbf{w}(t) - \mathbf{w}(s) \sim \mathcal{N}(0, t - s)$.
3. Independent increments:
For every $0 \leq s < t$, $\mathbf{w}(t) - \mathbf{w}(s)$ is independent of the past values.
4. Continuity: $\mathbf{w}(t)$ is continuous almost surely.



Variational Diffusion Models

Variational diffusion models have a **general** form with an arbitrary **forward diffusion (forward jump)** distribution $q(\mathbf{x}_t|\mathbf{x}_0)$:

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \alpha(t)\mathbf{x}_0, \sigma^2(t)\mathbf{I})$$

Now we follow the notations of Variational Diffusion Models

which satisfies the condition that **Signal-to-Noise Ratio (SNR)** $\frac{\alpha^2(t)}{\sigma^2(t)}$ monotonically **decreases** over time.

SDE $\rightarrow$ Diffusion Model

從 SDE 推出 diffusion 正向加噪

For a forward process SDE:

$$dx(t) = f(t)x(t)dt + g(t)d\mathbf{w}$$

its corresponding forward diffusion distribution is defined as:

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \alpha(t)\mathbf{x}_0, \sigma^2(t)\mathbf{I})$$

Forward jump 正向跳躍

- $\alpha(t) = e^{\int_0^t f(s)ds}$
- $\sigma^2(t) = \int_0^t \frac{g(s)^2}{\alpha(s)^2} ds$

Diffusion Model $\rightarrow$ SDE

從 diffusion 正向加噪推出 SDE

Given a **forward diffusion** distribution:

$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \alpha(t)\mathbf{x}_0, \sigma^2(t)\mathbf{I})$$

its corresponding SDE becomes:

$$d\mathbf{x}(t) = f(t)\mathbf{x}(t)dt + g(t)d\mathbf{w}$$

$$\bullet f(t) = \frac{d \log \alpha(t)}{dt} = \frac{d \log \alpha_t}{dt}$$

Diffusion / SDE 可以互相轉換

$$\bullet g(t)^2 = \frac{d\sigma^2(t)}{dt} - 2 \frac{d \log \alpha(t)}{dt} \sigma^2(t) = \frac{d\sigma_t^2}{dt} - 2 \frac{d \log \alpha_t}{dt} \sigma_t^2$$

Diffusion Model $\leftrightarrow$ SDE

Appendices

A Additional results

Figure 6 presents generated images for class-conditional ImageNet-64 [8] using the pre-trained ADM model by Dhariwal and Nichol [9]. The original DDM [47] and iDDPM [37] samplers are compared to ours in both deterministic and stochastic settings (Sections 3 and 4). Figure 7 shows the corresponding results that we obtain by training the model from scratch using our improved training configuration (Section 5).

The original samplers and training configurations by Song et al. [49] are compared to ours in Figures 8 and 9 (unconditional CIFAR-10 [29]), Figure 10 (class-conditional CIFAR-10), and Figure 11 (FFHQ [27] and AFHQv2 [7]). For ease of comparison, the same latent codes z_0 are used for each dataset/scenario across different training configurations and ODE choices. Figure 12 shows generated image quality with various NFE when using deterministic sampling.

Tables 3 and 4 summarize the numerical results on deterministic and stochastic sampling methods in various datasets, previously shown as functions of NFE in Figures 2 and 4.

B Derivation of formulas

B.1 Original ODE / SDE formulation from previous work

Song et al. [49] define their forward SDE (Eq. 5 in [49]) as

$$dx = f(x, t) dt + g(t) d\omega, \quad (9)$$

where ω is the standard Wiener process and $f(\cdot, t) : \mathbb{R}^d \rightarrow \mathbb{R}^d$ and $g(\cdot) : \mathbb{R} \rightarrow \mathbb{R}$ are the drift and diffusion coefficients, respectively, where d is the dimensionality of the dataset. These coefficients are selected differently for the variance preserving (VP) and variance exploding (VE) formulations, and $f(\cdot)$ is always of the form $f(x, t) = f(t)x$, where $f(\cdot) : \mathbb{R} \rightarrow \mathbb{R}$. Thus, the SDE can be equivalently written as

$$dx = f(t)x dt + g(t) d\omega. \quad (10)$$

The perturbation kernels of this SDE (Eq. 29 in [49]) have the general form

$$p_{\omega}(x(t) | x(0)) = N(x(t); s(t)x(0), s(t)^2 \sigma(t)^2 \mathbf{I}), \quad (11)$$

where $N(x; \mu, \Sigma)$ denotes the probability density function of $N(\mu, \Sigma)$ evaluated at x ,

$$s(t) = \exp\left(\int_0^t f(s) ds\right), \quad \text{and} \quad \sigma(t) = \sqrt{\int_0^t \frac{g(s)^2}{s(s)^2} ds}. \quad (12)$$

The marginal distribution $p_t(x)$ is obtained by integrating the perturbation kernels over $x(0)$:

$$p_t(x) = \int_{\mathbb{R}^d} p_{\omega}(x | x_0) p_{\omega_0}(x_0) dx_0. \quad (13)$$

Song et al. [49] define the probability flow ODE (Eq. 13 in [49]) so that it obeys this same $p_t(x)$:

$$dx = [f(t)x - \frac{1}{2} g(t)^2 \nabla_x \log p_t(x)] dt. \quad (14)$$

B.2 Our ODE formulation (Eq. 1 and Eq. 4)

The original ODE formulation (Eq. 14) is built around the functions f and g that correspond directly to specific terms that appear in the formula; the properties of the marginal distribution (Eq. 12) can only be derived indirectly based on these functions. However, f and g are of little practical interest in themselves, whereas the marginal distributions are of utmost importance in terms of training the model in the first place, bootstrapping the sampling process, and understanding how the ODE behaves in practice. Given that the idea of the probability flow ODE is to match a particular set of marginal

Let us now substitute Eq. 74 into Eq. 4, approximating the ideal denoiser $D(\cdot)$ with our trained model $D_{\theta}(\cdot)$:

$$dx = \left[\frac{s(t)}{\sigma(t)} x / s(t) - s(t)^2 \sigma(t) \sigma(t) \left[\frac{1}{\sigma(t)\sigma(t)} \left(D_{\theta}(x; \sigma(t)) - \hat{x} \right) \right] \right] dt \quad (75)$$

$$= \left[\frac{s(t)}{\sigma(t)} x - \frac{2s(t)g(t)}{\sigma(t)} \left(D_{\theta}(x; \sigma(t)) - \hat{x} \right) \right] dt. \quad (76)$$

Finally, backsubstitute $x = x/s(t)$:

$$dx = \left[\frac{s(t)}{\sigma(t)} x - \frac{2s(t)g(t)}{\sigma(t)} \left(D_{\theta}(\hat{x}; \sigma(t)) - [\hat{x}] \right) \right] dt \quad (77)$$

$$= \left[\frac{s(t)}{\sigma(t)} x - \frac{2s(t)g(t)}{\sigma(t)} \left(D_{\theta}([x/s(t)]; \sigma(t)) - [x/s(t)] \right) \right] dt \quad (78)$$

$$= \left[\frac{s(t)}{\sigma(t)} x - \frac{s(t)g(t)}{\sigma(t)} D_{\theta}(x/s(t); \sigma(t)) + \frac{2g(t)}{\sigma(t)} x \right] dt \quad (79)$$

$$= \left[\left(\frac{s(t)}{\sigma(t)} + \frac{2g(t)}{\sigma(t)} \right) x - \frac{s(t)g(t)}{\sigma(t)} D_{\theta}(x/s(t); \sigma(t)) \right] dt. \quad (80)$$

We can equivalently write Eq. 80 as

$$dx/dt = \left(\frac{\sigma(t)}{s(t)} + \frac{s(t)}{\sigma(t)} \right) x - \frac{\sigma(t)s(t)}{\sigma(t)} D_{\theta} \left(\frac{x}{s(t)}; \sigma(t) \right). \quad (81)$$

matching lines 4 and 7 of Algorithm 1.

B.5 Our SDE formulation (Eq. 6)

We derive the SDE of Eq. 6 by the following strategy:

- The desired marginal densities $p(x; \sigma(t))$ are convolutions of the data density p_{data} and an isotropic Gaussian density with standard deviation $\sigma(t)$ (see Eq. 20). Hence, considered as a function of the time t , the density evolves according to a heat diffusion PDE with time-varying diffusivity. As a first step, we find this PDE.
- We then use the Fokker-Planck equation to recover a family of SDEs for which the density evolves according to this PDE. Eq. 6 is obtained from a suitable parameterization of this family.

B.5.1 Generating the marginals by heat diffusion

We consider the time evolution of a probability density $q(x, t)$. Our goal is to find a PDE whose solution with the initial value $q(x, 0) := p_{\text{data}}(x)$ is $q(x, t) = p(x, \sigma(t))$. That is, the PDE should reproduce the marginals we postulate in Eq. 20.

The desired marginals are convolutions of p_{data} with isotropic normal distributions of time-varying standard deviation $\sigma(t)$, and as such, can be generated by the heat equation with time-varying diffusivity $\kappa(t)$. The situation is most conveniently analyzed in the Fourier domain, where the marginal densities are simply pointwise products of a Gaussian function and the transformed data density. To find the diffusivity that induces the correct standard deviations, we first write down the heat equation PDE:

$$\frac{\partial q(x, t)}{\partial t} = \kappa(t) \Delta_x q(x, t). \quad (82)$$

The Fourier transformed counterpart of Eq. 82, where the transform is taken along the x -dimension, is given by

$$\frac{\partial \hat{q}(\nu, t)}{\partial t} = -\kappa(t) |\nu|^2 \hat{q}(\nu, t). \quad (83)$$

The target solution $q(x, t)$ and its Fourier transform $\hat{q}(\nu, t)$ are given by Eq. 20:

$$q(x, t) = p(x; \sigma(t)) = p_{\text{data}}(x) * N(0, \sigma(t)^2 \mathbf{I}) \quad (84)$$

$$\hat{q}(\nu, t) = \hat{p}_{\text{data}}(\nu) \exp\left(-\frac{1}{2} |\nu|^2 \sigma(t)^2\right). \quad (85)$$

Differentiating the target solution along the time axis, we have

$$\frac{\partial \hat{q}(\nu, t)}{\partial t} = -\sigma(t) \sigma'(t) |\nu|^2 \hat{p}_{\text{data}}(\nu) \exp\left(-\frac{1}{2} |\nu|^2 \sigma(t)^2\right) \quad (86)$$

$$= -\sigma(t) \sigma'(t) |\nu|^2 \hat{q}(\nu, t). \quad (87)$$

Eqs. 83 and 87 share the same left hand side. Equating them allows us to solve for $\kappa(t)$ that generates the desired evolution:

$$-\kappa(t) |\nu|^2 \hat{q}(\nu, t) = -\sigma(t) \sigma'(t) |\nu|^2 \hat{q}(\nu, t) \quad (88)$$

$$\kappa(t) = \sigma(t) \sigma'(t). \quad (89)$$

To summarize, the desired marginal densities corresponding to noise levels $\sigma(t)$ are generated by the PDE

$$\frac{\partial q(x, t)}{\partial t} = \sigma(t) \sigma'(t) \Delta_x q(x, t) \quad (90)$$

from the initial density $q(x, 0) = p_{\text{data}}(x)$.

B.5.2 Derivation of our SDE

Given an SDE

$$dx = f(x, t) dt + g(x, t) d\omega, \quad (91)$$

the Fokker-Planck PDE describes the time evolution of its solution probability density $r(x, t)$ as

$$\frac{\partial r(x, t)}{\partial t} = -\nabla_x \cdot (f(x, t) r(x, t)) + \frac{1}{2} \nabla_x \nabla_x : (D(x, t) r(x, t)), \quad (92)$$

where $D_{ij} = \sum_k g_{ik} g_{jk}$ is the *diffusion tensor*. We consider the special case $g(x, t) = g(t) \mathbf{I}$ of x -independent white noise addition, whereby the equation simplifies to

$$\frac{\partial r(x, t)}{\partial t} = -\nabla_x \cdot (f(x, t) r(x, t)) + \frac{1}{2} g(t)^2 \Delta_x r(x, t). \quad (93)$$

We are seeking an SDE whose solution density is described by the PDE in Eq. 90. Setting $r(x, t) = q(x, t)$ and equating Eqs. 93 and 90, we find the sufficient condition that the SDE must satisfy

$$-\nabla_x \cdot (f(x, t) q(x, t)) + \frac{1}{2} g(t)^2 \Delta_x q(x, t) = \sigma(t) \sigma'(t) \Delta_x q(x, t) \quad (94)$$

$$\nabla_x \cdot (f(x, t) q(x, t)) = \left(\frac{1}{2} g(t)^2 - \sigma(t) \sigma'(t) \right) \Delta_x q(x, t). \quad (95)$$

Any choice of functions $f(x, t)$ and $g(t)$ satisfying this equation constitute a sought after SDE. Let us now find a specific family of such solutions. The key idea is given by the identity $\nabla_x \cdot \nabla_x = \Delta_x$. Indeed, if we set $f(x, t) q(x, t) = v(t) \nabla_x q(x, t)$ for any choice of $v(t)$, the term $\Delta_x q(x, t)$ appears on both sides and cancels out:

$$\nabla_x \cdot (v(t) \nabla_x q(x, t)) = \left(\frac{1}{2} g(t)^2 - \sigma(t) \sigma'(t) \right) \Delta_x q(x, t) \quad (96)$$

$$v(t) \Delta_x q(x, t) = \left(\frac{1}{2} g(t)^2 - \sigma(t) \sigma'(t) \right) \Delta_x q(x, t) \quad (97)$$

$$v(t) = \frac{1}{2} g(t)^2 - \sigma(t) \sigma'(t). \quad (98)$$

The stated $f(x, t)$ is in fact proportional to the score function, as the formula matches the gradient of the logarithm of the density:

$$f(x, t) = v(t) \frac{\nabla_x q(x, t)}{q(x, t)} \quad (99)$$

$$= v(t) \nabla_x \log q(x, t) \quad (100)$$

$$= \left(\frac{1}{2} g(t)^2 - \sigma(t) \sigma'(t) \right) \nabla_x \log q(x, t). \quad (101)$$

Substituting this back into Eq. 91 and writing $p(x; \sigma(t))$ in place of $q(x, t)$, we recover a family of SDEs whose solution densities have the desired marginals with noise levels $\sigma(t)$ for any choice of $g(t)$:

$$dx = \left(\frac{1}{2} g(t)^2 - \sigma(t) \sigma'(t) \right) \nabla_x \log p(x; \sigma(t)) dt + g(t) d\omega. \quad (102)$$

The free parameter $g(t)$ effectively specifies the rate of noise replacement at any given time instance. The special case choice of $g(t) = 0$ corresponds to the probability flow ODE. The parameterization by $g(t)$ is not particularly intuitive, however. To obtain a more interpretable parameterization, we set $g(t) = \sqrt{2\beta(t)} \sigma(t)$, which yields the (forward) SDE of Eq. 6 in the main paper:

$$dx = -\sigma(t) \sigma'(t) \nabla_x \log p(x; \sigma(t)) dt + \beta(t) \sigma(t) \nabla_x \log p(x; \sigma(t)) dt + \sqrt{2\beta(t)} \sigma(t) d\omega. \quad (103)$$

The noise replacement is now proportional to the standard deviation $\sigma(t)$ of the noise, with the proportionality factor $\beta(t)$. Indeed, expanding the score function in the middle term according to Eq. 3 yields $\beta(t) [D(x; \sigma(t)) - \hat{x}] dt$, which changes x proportionally to the negative noise component; the stochastic term injects new noise at the same rate. Intuitively, scaling the magnitude of Langevin exploration according to the current noise standard deviation is a reasonable baseline, as the data manifold is effectively "spread out" by this amount due to the blurring of the density.

The *reverse* SDE used in denoising diffusion is simply obtained by applying the time reversal formula of Anderson [1] (as stated in Eq. 6 of Song et al. [49]) on Eq. 103; the entire effect of the reversal is a change of sign in the middle term.

The scaled generalization of the SDE can be derived using a similar approach as with the ODE previously. As such, the derivation is omitted here.

B.6 Our preconditioning and training (Eq. 8)

Following Eq. 2, the denoising score matching loss for a given denoiser D_{θ} on a given noise level σ is given by

$$\mathcal{L}(D_{\theta}; \sigma) = \mathbb{E}_{p_{\text{data}}, \mathbb{E}_{x \sim N(0, \sigma^2 \mathbf{I})}} \|D_{\theta}(y + \pi; \sigma) - y\|_2^2 \quad (104)$$

We obtain overall training loss by taking a weighted expectation of $\mathcal{L}(D_{\theta}; \sigma)$ over the noise levels:

$$\mathcal{L}(D_{\theta}) = \mathbb{E}_{x \sim p_{\text{data}}} [\lambda(\sigma) \mathcal{L}(D_{\theta}; \sigma)] \quad (105)$$

$$= \mathbb{E}_{x \sim p_{\text{data}}} [\lambda(\sigma) \mathbb{E}_{y \sim p_{\text{data}} | x} \|D_{\theta}(y + \pi; \sigma) - y\|_2^2] \quad (106)$$

$$= \mathbb{E}_{x \sim p_{\text{data}}} \mathbb{E}_{y \sim p_{\text{data}} | x} \mathbb{E}_{\pi \sim N(0, \sigma^2 \mathbf{I})} [\lambda(\sigma) \|D_{\theta}(y + \pi; \sigma) - y\|_2^2] \quad (107)$$

$$= \mathbb{E}_{x \sim p_{\text{data}}} [\lambda(\sigma) \|D_{\theta}(y + \pi; \sigma) - y\|_2^2], \quad (108)$$

where the noise levels are distributed according to $\sigma \sim p_{\text{data}}$ and weighted by $\lambda(\sigma)$.

Using our definition of $D_{\theta}(\cdot)$ from Eq. 7, we can further rewrite $\mathcal{L}(D_{\theta})$ as

$$\mathbb{E}_{x \sim p_{\text{data}}} [\lambda(\sigma) \|c_{\text{data}}(\sigma)(y + \pi) + c_{\text{data}}(\sigma) F_{\theta}(c_{\text{data}}(\sigma)(y + \pi); c_{\text{data}}(\sigma)) - y\|_2^2] \quad (109)$$

$$= \mathbb{E}_{x \sim p_{\text{data}}} [\lambda(\sigma) \|c_{\text{data}}(\sigma) F_{\theta}(c_{\text{data}}(\sigma)(y + \pi); c_{\text{data}}(\sigma)) - (y - c_{\text{data}}(\sigma)(y + \pi))\|_2^2] \quad (110)$$

$$= \mathbb{E}_{x \sim p_{\text{data}}} [\lambda(\sigma) c_{\text{data}}(\sigma)^2 \|F_{\theta}(c_{\text{data}}(\sigma)(y + \pi); c_{\text{data}}(\sigma)) - \frac{1}{c_{\text{data}}(\sigma)} (y - c_{\text{data}}(\sigma)(y + \pi))\|_2^2] \quad (111)$$

$$= \mathbb{E}_{x \sim p_{\text{data}}} [v(\sigma) \|F_{\theta}(c_{\text{data}}(\sigma)(y + \pi); c_{\text{data}}(\sigma)) - F_{\text{target}}(y; \pi; \sigma)\|_2^2], \quad (112)$$

which matches Eq. 8 and corresponds to traditional supervised training of F_{θ} using standard L_2 loss with effective weight $v(\cdot)$ and target $F_{\text{target}}(\cdot)$ given by

$$v(\sigma) = \lambda(\sigma) c_{\text{data}}(\sigma)^2 \quad \text{and} \quad F_{\text{target}}(y; \pi; \sigma) = \frac{1}{c_{\text{data}}(\sigma)} (y - c_{\text{data}}(\sigma)(y + \pi)). \quad (113)$$

We can now derive formulas for $c_{\text{data}}(\sigma)$, $c_{\text{data}}(\sigma)$, $c_{\text{data}}(\sigma)$, and $\lambda(\sigma)$ from first principles, shown in the "Ops" column of Table 1.

Diffusion Model $\rightarrow$ SDE

$$\begin{aligned} g(s)^2 &= \frac{d\sigma_t^2}{dt} - 2 \frac{d \log \alpha_t}{dt} \sigma_t^2 \\ &= 2\sigma_t \frac{d\sigma_t}{dt} - 2\sigma_t^2 \frac{d \log \alpha_t}{dt} \\ &= 2\sigma_t \left(\sigma_t \frac{d \log \sigma_t}{dt} \right) - 2\sigma_t^2 \frac{d \log \alpha_t}{dt} \\ &= 2\sigma_t^2 \left(\frac{d \log \sigma_t}{dt} - \frac{d \log \alpha_t}{dt} \right) \\ &= 2\sigma_t^2 \left(-\frac{d\lambda_t}{dt} \right) \quad \text{where } \lambda_t = \log \frac{\alpha_t}{\sigma_t}, \end{aligned}$$

這個 λ_t 後面會一直出現
記得他是隨時間遞減的

which is **decreasing** over time.

Recall: VP-SDE vs. VE-SDE

- **DDPM** is a special case (**Variance Preserving**) when the forward transitional distribution $q(\mathbf{x}_t|\mathbf{x}_{t-1})$ is defined as

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I})$$

- **SMLD** (Score Matching with Langevin Dynamics) is another case (**Variance Exploding**) when the forward transitional distribution $q(\mathbf{x}_t|\mathbf{x}_{t-1})$ is:

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \mathbf{x}_{t-1}, (\sigma_t^2 - \sigma_{t-1}^2)\mathbf{I})$$

Variational Diffusion Models

Variational diffusion models have a **general** form with an arbitrary **forward diffusion (forward jump)** distribution $q(\mathbf{x}_t|\mathbf{x}_0)$:

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \alpha(t)\mathbf{x}_0, \sigma^2(t)\mathbf{I})$$

Now we follow the notations of Variational Diffusion Models

which satisfies the condition that **Signal-to-Noise Ratio (SNR)** $\frac{\alpha^2(t)}{\sigma^2(t)}$ monotonically **decreases** over time.

有非常多種方式可以定義 diffusion forward process (例如前一頁的 DDIM 或 SMLD)
只要符合這個 SNR 隨時間遞減的條件即可

Variance Preserving (VP) SDE

- In the **continuous** setup of Variance Preserving (VP) SDE:

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N} \left(\mathbf{x}_t; \sqrt{1 - \beta(t)} \mathbf{x}_{t-1}, \beta(t) \mathbf{I} \right)$$

因為現在在 continuous time domain，所以 β_t 也變成 $\beta(t)$ ，但仍是 pre-defined

- The **forward jump** distribution becomes:

$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \alpha(t) \mathbf{x}_0, \sigma^2(t) \mathbf{I})$$

where $\alpha(t) = e^{-\frac{1}{2} \int_0^t \beta(s) ds}$ and $\sigma^2(t) = 1 - \alpha(t)$.

Variance Preserving (VP) SDE

$$\alpha(t) = e^{-\frac{1}{2} \int_0^t \beta(s) ds} \quad \sigma^2(t) = 1 - \alpha(t)$$

Note that $0 \leq \beta_1 \leq \beta_2 \leq \dots \beta_T$.

Q. Does the Signal-to-Noise Ratio (SNR) $\frac{\alpha^2(t)}{\sigma^2(t)}$ decrease over time?

正確的 diffusion 要符合這個 SNR 隨時間遞減的條件

Variance Preserving (VP) SDE

$$\alpha(t) = e^{-\frac{1}{2} \int_0^t \beta(s) ds} \quad \sigma^2(t) = 1 - \alpha(t)$$

- $f(t) = \frac{d \log \alpha(t)}{dt} = \frac{d \log \alpha_t}{dt}$
- $g^2(t) = \frac{d\sigma^2(t)}{dt} - 2 \frac{d \log \alpha(t)}{dt} \sigma^2(t)$

Q. Describe $f(t)$ and $g^2(t)$ of the forward process SDE with respect to $\beta(t)$.

Variance Preserving (VP) SDE

Given the forward transitional distribution of the VP-SDE case:

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}\left(\mathbf{x}_t; \sqrt{1 - \beta(t)}\mathbf{x}_{t-1}, \beta(t)\mathbf{I}\right)$$

The corresponding SDE for the forward process is defined as

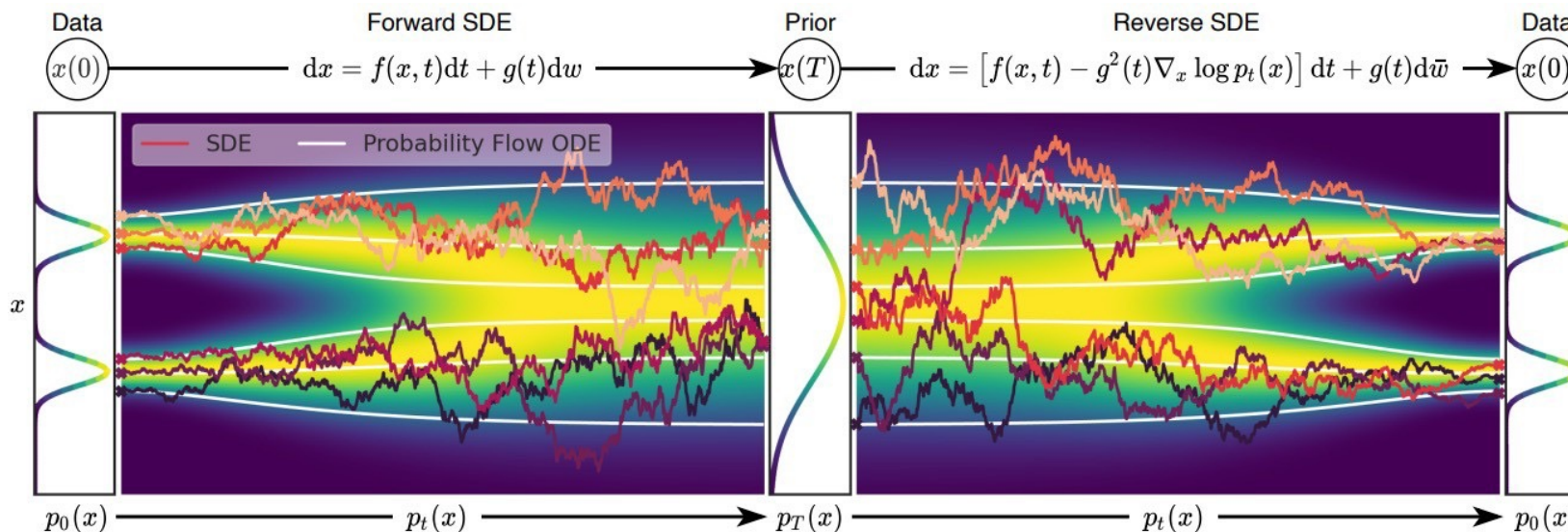
$$dx(t) = \underbrace{-\frac{1}{2}\beta(t)x(t)dt}_{f(t)} + \underbrace{\sqrt{\beta(t)}d\mathbf{w}}_{g(t)}, \quad x(0) \sim q(\mathbf{x}_0)$$

DDPM 對應的 SDE，所以如果我們會解這個 **SDE**，那我們就可以找到 **trajectory** $x(t)$

Probability Flow ODE

A SDE has a corresponding ODE (**no Wiener process**) that has same marginal distribution at every time step.

- **Red curves: SDE** 因為在每個 time step，SDE 與對應的 ODE 有相同的 marginal distribution 與其去解 trajectory 崎嶇難解的 SDE，不如去解對應的 ODE
- **White curves: ODE**



Probability Flow ODE

Variance Preserving (VP) SDE

$$\alpha(t) = e^{-\frac{1}{2} \int_0^t \beta(s) ds} \quad \sigma^2(t) = 1 - \alpha(t)$$

$$f(t) = \frac{\frac{d \log \alpha(t)}{dt}}{\frac{d \log \alpha_t}{dt}}$$

$$g^2(t) = \frac{d\sigma^2(t)}{dt} - 2 \frac{d \log \alpha(t)}{dt} \sigma^2(t)$$

Reverse process **SDE**:

$$dx(t) = \left[f(t)x(t) + \frac{g^2(t)}{\sqrt{1 - \alpha_t}} \hat{\epsilon}_\theta(x(t), t) \right] dt + g(t) d\bar{\mathbf{w}}_t$$

(Note: In the original image, an orange arrow points from the label σ_t to the denominator $\sqrt{1 - \alpha_t}$.)

The corresponding probability flow **ODE**:

$$\frac{dx(t)}{dt} = f(t)x(t) + \frac{g^2(t)}{2\sigma_t} \hat{\epsilon}_\theta(x(t), t)$$

沒有後面那個 Wiener 隨機擾動項

Probability Flow ODE

Variance Preserving (VP) SDE

$$\alpha(t) = e^{-\frac{1}{2} \int_0^t \beta(s) ds} \quad \sigma^2(t) = 1 - \alpha(t)$$

$$f(t) = \frac{d \log \alpha(t)}{dt} = \frac{d \log \alpha_t}{dt}$$

$$g^2(t) = \frac{d\sigma^2(t)}{dt} - 2 \frac{d \log \alpha(t)}{dt} \sigma^2(t)$$

Plug $f(t) = \frac{d \log \alpha_t}{dt}$ and $g^2(t) = 2\sigma_t^2 \left(-\frac{d\lambda_t}{dt}\right)$ for the variance preserving case:

$$\frac{dx(t)}{dt} = \left(\frac{d \log \alpha_t}{dt}\right) x(t) + \sigma_t \frac{d\lambda_t}{dt} \hat{\epsilon}_\theta(x(t), t)$$

ODE 的解與 boundary condition

- 對任何 ODE，通常有無窮多個函數滿足這個方程式，這些函數形成一個 family of solutions
- 要從無窮多個解中選出一個，需要 boundary condition，可以指定起點、終點，或路徑上的任何一點
- 在 Reverse process 中：從指定的 noise $x(T)$ ，經 ODE 求解，得到唯一的 trajectory 走到真實資料分布 $x(0)$

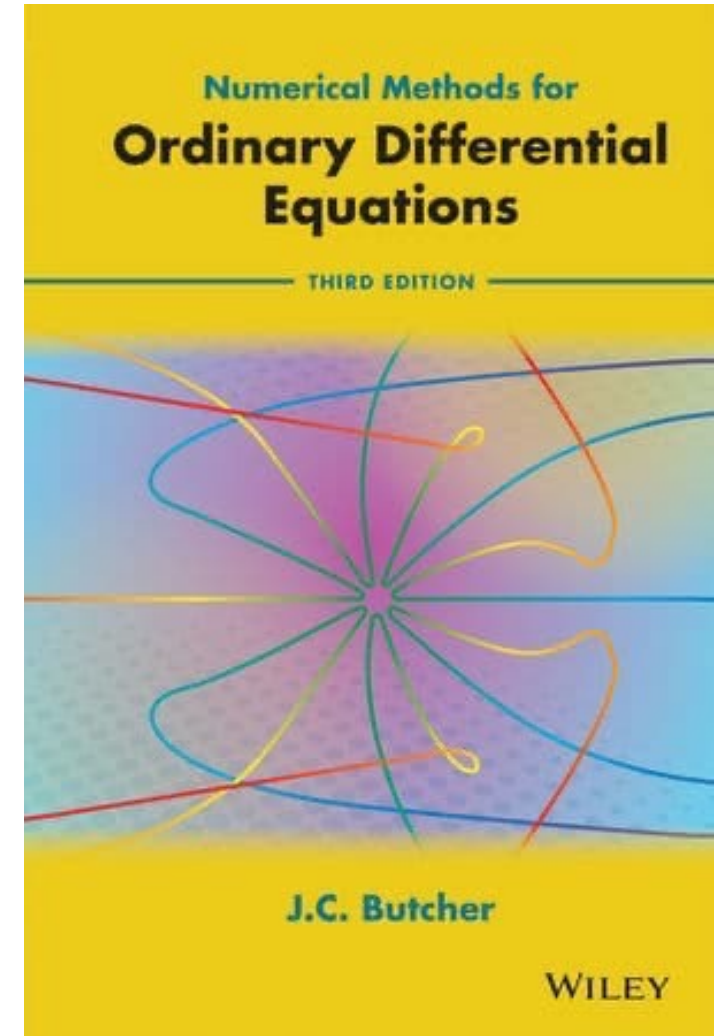
VP-ODE 這個方程式定義了一個從 noise distribution 到真實資料 distribution 的 deterministic mapping，只要你給定一個起始的 noise $x(T)$ ，ODE solver 就能沿著唯一的軌跡找到對應清晰影像 $x(0)$

Summary

1. The forward diffusion process can be described by an **SDE** in the continuous domain.
2. The **reverse** process of an SDE can also be formulated as another SDE.
3. A diffusion process SDE has a corresponding **ODE** that has the same marginal distribution at each time step.
4. Hence, the reverse diffusion (generation) process can be conducted by solving the corresponding ODE.

Numerical Methods for ODE

- Euler method
- Implicit (Backward) Euler method
- Midpoint Method (Second-order Runge-Kutta)
- Runge-Kutta Methods (RK4)
 - Runge-Kutta-Fehlberg Method (RKF45)
 - Diagonally Implicit Runge–Kutta (DIRK)
- Explicit multistep method:
 - Adams–Bashforth
- Implicit multistep methods:
 - Adams-Moulton
 - Backward Differentiation



Numerical Methods for ODE

- **General-purpose** ODE solvers require many steps for accurate results.
- Is there a **dedicated** solver for the following diffusion model ODE that can solve it accurately in just a **few steps**?

$$\frac{dx(t)}{dt} = \left(\frac{d \log \alpha_t}{dt} \right) x(t) + \sigma_t \frac{d\lambda_t}{dt} \hat{\epsilon}_{\theta}(x(t), t)$$

既然我們都知道 ODE 就是這個特定的形式，那我們能不能不用 general ODE solvers，而是推導出一個專用的 solver 來解這個 ODE 呢？

Ordinary Differential Equation

First-Order Linear Differential Equation

First-order linear differential equation has the following general form:

$$\underset{\text{First-order derivative}}{x'(t)} = \overset{\text{Linear}}{p(t)x(t)} + q(t)$$

↓
A unknown function of t .

Note that “linear” does NOT mean that $p(t)$ is linear, but the equation is linear in x and its derivative with respect to t .

First-Order Linear Differential Equation

Let's first think about a special when $q(t) = 0$.

$$x'(t) = p(t)x(t)$$

How to find a function $x(t)$ that satisfies this equation?

First-Order Linear Differential Equation

When assuming $x(t) \neq 0$ for all t ,

$$\frac{1}{x(t)} x'(t) = p(t)$$

What is the simplified form of the expression $\frac{1}{x(t)} x'(t)$ based on calculus techniques?

First-Order Linear Differential Equation

When assuming $x(t) \neq 0$ for all t ,

$$\frac{1}{x(t)} x'(t) = \frac{d \ln x(t)}{dt} = p(t)$$

Integrate both sides of the equation, and then take the **exponential**.

First-Order Linear Differential Equation

$$\ln x(t) = \ln x(t_0) + \int_{t_0}^t p(\tau) d\tau$$

$$\Rightarrow x(t) = e^{\ln x(t_0) + \int_{t_0}^t p(\tau) d\tau}$$

$$= e^{\ln x(t_0)} \cdot e^{\int_{t_0}^t p(\tau) d\tau}$$

$$= C \cdot e^{\int p(\tau) d\tau}$$

First-Order Linear Differential Equation

Q. Find a function $x(t)$ that satisfies the following ordinary differential equation:

$$x'(t) = -\frac{1}{2}x(t)$$

and the boundary condition:

$$x(0) = 1$$

First-Order Linear Differential Equation

A.

$$x(t) = e^{-\frac{1}{2}t}$$

First-Order Linear Differential Equation

Based on the solution for the **special** case:

$$x'(t) = p(t)x(t)$$

can we find the solution for the **general** case:

$$x'(t) = p(t)x(t) + q(t)$$

First-Order **Semilinear** Differential Equation

Let's think about an even more general case:

$$x'(t) = \overset{\text{Linear}}{p(t)x(t)} + q(x(t), t)$$

↓
This function may NOT be
linear with respect to $x(t)$.

Such a function is called a first-order **semilinear** ordinary differential equation.

First-Order Semilinear Differential Equation

How to solve it?

Recall that the solution of the following ODE:

$$x'(t) = p(t)x(t)$$

is given by:

$$x(t) = C \cdot e^{\int p(\tau) d\tau}$$

First-Order Semilinear Differential Equation

Exponential Integrator

Given first-order **semilinear** ODE:

$$x'(t) - p(t)x(t) = q(x(t), t)$$

別問為什麼要乘 & 為什麼是乘這一項，乘就對了！

Inspired by the solution of the special case, multiply $e^{-\int_{t_0}^t p(r)dr}$ sides:

$$e^{-\int_{t_0}^t p(r)dr} x'(t) - e^{-\int_{t_0}^t p(r)dr} p(t)x(t) = e^{-\int_{t_0}^t p(r)dr} q(x(t), t)$$

First-Order Semilinear Differential Equation

$$e^{-\int_{t_0}^t p(r)dr} x'(t) - e^{-\int_{t_0}^t p(r)dr} p(t)x(t) = e^{-\int_{t_0}^t p(r)dr} q(x(t), t)$$

Note that $\frac{d}{dt} \left(e^{-\int_{t_0}^t p(r)dr} \right) = -p(t)e^{-\int_{t_0}^t p(r)dr}$.

Simplify the left side.

Hint. $\frac{d}{dt} (f(t)g(t)) = f'(t)g(t) + f(t)g'(t)$

First-Order Semilinear Differential Equation

$$\begin{aligned} e^{-\int_{t_0}^t p(r)dr} x'(t) - e^{-\int_{t_0}^t p(r)dr} p(t)x(t) \\ = \frac{d}{dt} \left(e^{-\int_{t_0}^t p(r)dr} x(t) \right) = e^{-\int_{t_0}^t p(r)dr} q(x(t), t) \end{aligned}$$

Simplify the left side.

Hint. $\frac{d}{dt} (f(t)g(t)) = f'(t)g(t) + f(t)g'(t)$

First-Order Semilinear Differential Equation

$$\frac{d}{dt} \left(e^{-\int_{t_0}^t p(r) dr} x(t) \right) = e^{-\int_{t_0}^t p(r) dr} q(x(t), t)$$

$$\Rightarrow e^{-\int_{t_0}^t p(r) dr} x(t) = e^{-\int_{t_0}^s p(r) dr} x(s) + \int_s^t e^{-\int_{t_0}^{\tau} p(r) dr} q(x(\tau), \tau) d\tau$$

$t < s$

$$\Rightarrow x(t) = e^{\int_{t_0}^t p(r) dr - \int_{t_0}^s p(r) dr} x(s) + \int_s^t e^{\int_{t_0}^t p(r) dr - \int_{t_0}^{\tau} p(r) dr} q(x(\tau), \tau) d\tau$$

First-Order Semilinear Differential Equation

$$x(t) = e^{\int_{t_0}^t p(r)dr - \int_{t_0}^s p(r)dr} x(s) + \int_s^t e^{\int_{t_0}^t p(r)dr - \int_{t_0}^\tau p(r)dr} q(x(\tau), \tau) d\tau$$

$$x(t) = e^{\int_s^t p(r)dr} x(s) + \int_s^t e^{\int_\tau^t p(r)dr} q(x(\tau), \tau) d\tau$$

First-Order Linear Differential Equation

Q. Find a function $x(t)$ that satisfies the following ordinary differential equation:

$$x'(t) = -\frac{1}{2}x(t) + e^{-\frac{1}{2}t}$$

and the boundary condition:

$$x(0) = 1$$

First-Order Semilinear Differential Equation

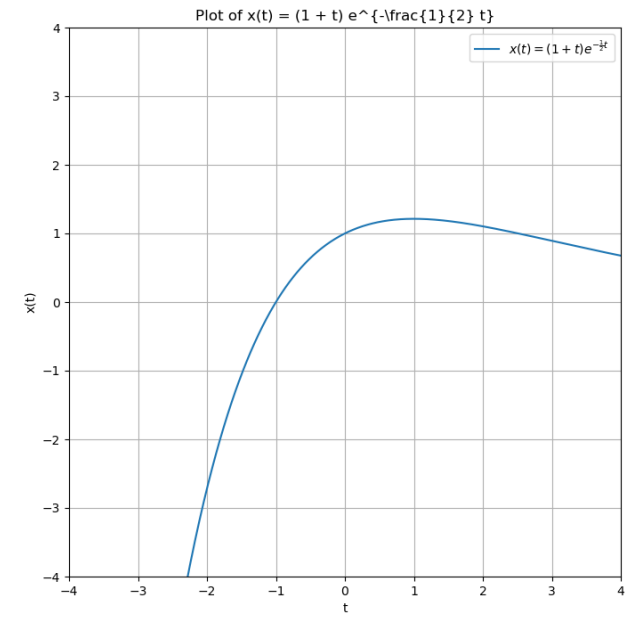
$$x(t) = e^{\int_{t_0}^t p(r)dr - \int_{t_0}^s p(r)dr} x(s) + \int_s^t e^{\int_{t_0}^t p(r)dr - \int_{t_0}^\tau p(r)dr} q(x(\tau), \tau) d\tau$$

$$x(t) = e^{\int_s^t p(r)dr} x(s) + \int_s^t e^{\int_\tau^t p(r)dr} q(x(\tau), \tau) d\tau$$

First-Order Linear Differential Equation

A. Let $s = 0$. $x(s) = 1$.

$$\begin{aligned} x(t) &= e^{-\frac{1}{2}t} + \int_s^t e^{-\frac{1}{2}(t-\tau)} e^{-\frac{1}{2}\tau} d\tau \\ &= (1+t)e^{-\frac{1}{2}t} \end{aligned}$$



所以我們可以解出 First-Order Linear Differential Equation 的 $p(t)$ 與 $q(x(t), t)$

DPM-Solver

Lu et al., DPM-Solver: A Fast ODE Solver for Diffusion Probabilistic Model Sampling in Around 10 Steps, NeurIPS 2022.

Coming Back to PF-ODE...

The reverse process ODE is a **first-order semilinear** ODE:

$$x'(t) = \underbrace{\left(\frac{d \log \alpha_t}{dt} \right)}_{p(t)} x(t) - \underbrace{\sigma_t \frac{d\lambda_t}{dt} \hat{\epsilon}_\theta(x(t), t)}_{q(x(t), t)}$$

First-Order Semilinear Differential Equation

$$x(t) = e^{\int_{t_0}^t p(r)dr - \int_{t_0}^s p(r)dr} x(s) + \int_s^t e^{\int_{t_0}^t p(r)dr - \int_{t_0}^\tau p(r)dr} q(x(\tau), \tau) d\tau$$

$$x(t) = e^{\int_s^t p(r)dr} x(s) + \int_s^t e^{\int_\tau^t p(r)dr} q(x(\tau), \tau) d\tau$$

DPM-Solver

$$x(t) = e^{\int_s^t p(r)dr} x(s) + \int_s^t e^{\int_\tau^t p(r)dr} q(x(\tau), \tau) d\tau$$

Plug $p(t) = \frac{d \log \alpha_t}{dt}$ and $q(x(t), t) = -\sigma_t \frac{d\lambda_t}{dt} \hat{\epsilon}_\theta(x(t), t)$

$$x(t) = \frac{\alpha_t}{\alpha_s} x(s) - \int_s^t \left(\frac{\alpha_t}{\alpha_\tau} \right) \sigma_\tau \frac{d\lambda_\tau}{d\tau} \hat{\epsilon}_\theta(x(\tau), \tau) d\tau$$

$$= \frac{\alpha_t}{\alpha_s} x(s) - \alpha_t \int_s^t \left(\frac{d\lambda_\tau}{d\tau} \right) e^{-\lambda_\tau} \hat{\epsilon}_\theta(x(\tau), \tau) d\tau$$

$$\because \frac{\alpha_\tau}{\sigma_\tau} = e^{-\lambda_\tau}$$

DPM-Solver

$\lambda_t = \lambda(t)$ is a **strictly decreasing** function of t .

$\Rightarrow$ There exists an **inverse** function $t_\lambda(\lambda_t) = t$.

For the second integral term, change the parameter $t \rightarrow \lambda$.

$$\begin{aligned} x(t) &= \frac{\alpha_t}{\alpha_s} x(s) - \alpha_t \int_s^t \left(\frac{d\lambda_\tau}{d\tau} \right) e^{-\lambda_\tau} \hat{\boldsymbol{\epsilon}}_\theta(x(\tau), \tau) d\tau \\ &= \frac{\alpha_t}{\alpha_s} x(s) - \alpha_t \int_s^t \left(\frac{d\lambda_\tau}{\cancel{d\lambda_\tau}} \right) e^{-\lambda_\tau} \hat{\boldsymbol{\epsilon}}_\theta(x(\lambda_\tau), \lambda_\tau) d\lambda_\tau \end{aligned}$$

DPM-Solver

In a **continuous** setup,

$$x(t) = \frac{\alpha_t}{\alpha_s} x(s) - \alpha_t \int_{\lambda_s}^{\lambda_t} e^{-\lambda} \hat{\boldsymbol{\varepsilon}}_{\theta}(x(\lambda), \lambda) d\lambda$$

$t < s$

In a **discrete** setup (let $s = t + 1$),

$$\boldsymbol{x}_t = \frac{\alpha_t}{\alpha_{t+1}} \boldsymbol{x}_{t+1} - \alpha_t \int_{\lambda_{t+1}}^{\lambda_t} e^{-\lambda} \hat{\boldsymbol{\varepsilon}}_{\theta}(x(\lambda), \lambda) d\lambda$$

*How can this be approximated
with discretized time steps?*

DPM-Solver-1

First-Order Approximation

In the discrete setup, assume that $\hat{\boldsymbol{\epsilon}}_{\theta}(x(\lambda), \lambda)$ is **constant** as $\hat{\boldsymbol{\epsilon}}_{\theta}(x(\lambda_{t+1}), \lambda_{t+1})$ in the range of $(\lambda_t, \lambda_{t+1}]$:

$$\begin{aligned}\mathbf{x}_t &\approx \frac{\alpha_t}{\alpha_{t+1}} \mathbf{x}_{t+1} - \alpha_t \hat{\boldsymbol{\epsilon}}_{\theta}(x(\lambda_{t+1}), \lambda_{t+1}) \int_{\lambda_{t+1}}^{\lambda_t} e^{-\lambda} d\lambda \\ &= \frac{\alpha_t}{\alpha_{t+1}} \mathbf{x}_{t+1} - \alpha_t (-e^{-\lambda_t} + e^{-\lambda_{t+1}}) \hat{\boldsymbol{\epsilon}}_{\theta}(x(\lambda_{t+1}), \lambda_{t+1})\end{aligned}$$

DPM-Solver-1

- Let $h_t := \lambda_t - \lambda_{t+1}$
- $-e^{\lambda_t} + e^{\lambda_{t+1}} = e^{\lambda_t}(e^{\lambda_t - \lambda_{t+1}} - 1) = \frac{\sigma_t}{\alpha_t}(e^{h_t} - 1)$

$$\because \frac{\sigma_t}{\alpha_t} = e^{\lambda_t}$$

$$\begin{aligned} \mathbf{x}_t &\approx \frac{\alpha_t}{\alpha_{t+1}} \mathbf{x}_{t+1} - \alpha_t (-e^{-\lambda_t} + e^{-\lambda_{t+1}}) \hat{\boldsymbol{\epsilon}}_{\theta}(\mathbf{x}(\lambda_{t+1}), \lambda_{t+1}) \\ &= \frac{\alpha_t}{\alpha_{t+1}} \mathbf{x}_{t+1} - \alpha_t \left(\frac{\sigma_{t+1}}{\alpha_{t+1}} - \frac{\sigma_t}{\alpha_t} \right) \hat{\boldsymbol{\epsilon}}_{\theta}(\mathbf{x}(\lambda_{t+1}), \lambda_{t+1}) \\ &= \frac{\alpha_t}{\alpha_{t+1}} \mathbf{x}_{t+1} - \left(\alpha_t \frac{\sigma_{t+1}}{\alpha_{t+1}} - \sigma_t \right) \hat{\boldsymbol{\epsilon}}_{\theta}(\mathbf{x}(\lambda_{t+1}), \lambda_{t+1}) \\ &= \frac{\alpha_t}{\alpha_{t+1}} \mathbf{x}_{t+1} - \sigma_t (e^{h_t} - 1) \hat{\boldsymbol{\epsilon}}_{\theta}(\mathbf{x}(\lambda_{t+1}), \lambda_{t+1}) \end{aligned}$$

用這種 solver 來做 reverse process，且沒有任何隨機性

Variance Preserving (VP) SDE

$$\alpha(t) = e^{-\frac{1}{2} \int_0^t \beta(s) ds} \quad \sigma^2(t) = 1 - \alpha(t)$$

$$\mathbf{x}_t \approx \frac{\alpha_t}{\alpha_{t+1}} \mathbf{x}_{t+1} - \left(\alpha_t \frac{\sigma_{t+1}}{\alpha_{t+1}} - \sigma_t \right) \hat{\boldsymbol{\epsilon}}_{\theta}(\mathbf{x}(\lambda_{t+1}), \lambda_{t+1})$$

Q. What is $\mathbf{x}_t$ with respect to $\mathbf{x}_{t+1}$ and $\beta(t)$?

DPM-Solver-1

- Let $h_t := \lambda_t - \lambda_{t+1}$
- $-e^{\lambda_t} + e^{\lambda_{t+1}} = e^{\lambda_t}(e^{\lambda_t - \lambda_{t+1}} - 1) = \frac{\sigma_t}{\alpha_t}(e^{h_t} - 1)$

$$\because \frac{\sigma_t}{\alpha_t} = e^{\lambda_t}$$

$$\begin{aligned} \mathbf{x}_t &\approx \frac{\alpha_t}{\alpha_{t+1}} \mathbf{x}_{t+1} - \alpha_t (-e^{-\lambda_t} + e^{-\lambda_{t+1}}) \hat{\boldsymbol{\epsilon}}_{\theta}(\mathbf{x}(\lambda_{t+1}), \lambda_{t+1}) \\ &= \frac{\alpha_t}{\alpha_{t+1}} \mathbf{x}_{t+1} - \alpha_t \left(\frac{\sigma_{t+1}}{\alpha_{t+1}} - \frac{\sigma_t}{\alpha_t} \right) \hat{\boldsymbol{\epsilon}}_{\theta}(\mathbf{x}(\lambda_{t+1}), \lambda_{t+1}) \\ &= \frac{\alpha_t}{\alpha_{t+1}} \mathbf{x}_{t+1} - \left(\alpha_t \frac{\sigma_{t+1}}{\alpha_{t+1}} - \sigma_t \right) \hat{\boldsymbol{\epsilon}}_{\theta}(\mathbf{x}(\lambda_{t+1}), \lambda_{t+1}) \\ &= \frac{\alpha_t}{\alpha_{t+1}} \mathbf{x}_{t+1} - \sigma_t (e^{h_t} - 1) \hat{\boldsymbol{\epsilon}}_{\theta}(\mathbf{x}(\lambda_{t+1}), \lambda_{t+1}) \end{aligned}$$

用這種 solver 來做 reverse process，且沒有任何隨機性

Variance Preserving (VP) SDE

$$\alpha(t) = e^{-\frac{1}{2} \int_0^t \beta(s) ds} \quad \sigma^2(t) = 1 - \alpha(t)$$

$$\mathbf{x}_t \approx \frac{\alpha_t}{\alpha_{t+1}} \mathbf{x}_{t+1} - \left(\alpha_t \frac{\sigma_{t+1}}{\alpha_{t+1}} - \sigma_t \right) \hat{\boldsymbol{\epsilon}}_{\theta}(\mathbf{x}(\lambda_{t+1}), \lambda_{t+1})$$

Q. What is $\mathbf{x}_t$ with respect to $\mathbf{x}_{t+1}$ and $\beta(t)$?

Recall: DDIM **Deterministic** Reverse Process

$$q_{\sigma}(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}\left(\sqrt{\bar{\alpha}_{t-1}}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \cdot \frac{\mathbf{x}_t - \sqrt{\bar{\alpha}_t}\mathbf{x}_0}{\sqrt{1 - \bar{\alpha}_t}}, \sigma_t^2 \mathbf{I}\right)$$

For each time step $t = T, \dots, 1$, repeat:

1. $\boldsymbol{\varepsilon}_t = \hat{\boldsymbol{\varepsilon}}_{\theta}(\mathbf{x}_t, t)$
2. $\mathbf{x}_{0|t} = \frac{1}{\sqrt{\bar{\alpha}_t}}(\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t}\boldsymbol{\varepsilon}_t)$
3. $\mathbf{x}_{t-1} = \sqrt{\bar{\alpha}_{t-1}}\mathbf{x}_{0|t} + \sqrt{1 - \bar{\alpha}_{t-1}}\boldsymbol{\varepsilon}_t$

Recall: DDIM **Deterministic** Reverse Process

$$\begin{aligned} \mathbf{x}_t &= \sqrt{\bar{\alpha}_t} \left(\frac{1}{\sqrt{\bar{\alpha}_{t+1}}} \left(\mathbf{x}_{t+1} - \sqrt{1 - \bar{\alpha}_{t+1}} \hat{\boldsymbol{\epsilon}}_{\theta}(\mathbf{x}_{t+1}, t+1) \right) \right) + \sqrt{1 - \bar{\alpha}_t} \hat{\boldsymbol{\epsilon}}_{\theta}(\mathbf{x}_{t+1}, t+1) \\ &= \frac{\sqrt{\bar{\alpha}_t}}{\sqrt{\bar{\alpha}_{t+1}}} \mathbf{x}_{t+1} - \left(\frac{\sqrt{\bar{\alpha}_t}}{\sqrt{\bar{\alpha}_{t+1}}} \sqrt{1 - \bar{\alpha}_{t+1}} - \sqrt{1 - \bar{\alpha}_t} \right) \hat{\boldsymbol{\epsilon}}_{\theta}(\mathbf{x}_{t+1}, t+1) \\ &= \frac{\alpha_t}{\alpha_{t+1}} \mathbf{x}_{t+1} + \left(\alpha_t \frac{\sigma_{t+1}}{\alpha_{t+1}} - \sigma_t \right) \hat{\boldsymbol{\epsilon}}_{\theta}(\mathbf{x}_{t+1}, t+1) \end{aligned}$$

$$\begin{aligned} \alpha_t &= \sqrt{\bar{\alpha}_t} \\ \sigma_t &= \sqrt{1 - \bar{\alpha}_t} \end{aligned}$$

The first-order approximation is identical to DDIM!

DPM-Solver-1

- Let $h_t := \lambda_t - \lambda_{t+1}$
- $-e^{\lambda_t} + e^{\lambda_{t+1}} = e^{\lambda_t}(e^{\lambda_t - \lambda_{t+1}} - 1) = \frac{\sigma_t}{\alpha_t}(e^{h_t} - 1)$

$$\because \frac{\sigma_t}{\alpha_t} = e^{\lambda_t}$$

$$\begin{aligned} \mathbf{x}_t &\approx \frac{\alpha_t}{\alpha_{t+1}} \mathbf{x}_{t+1} - \alpha_t (-e^{-\lambda_t} + e^{-\lambda_{t+1}}) \hat{\boldsymbol{\epsilon}}_{\theta}(\mathbf{x}(\lambda_{t+1}), \lambda_{t+1}) \\ &= \frac{\alpha_t}{\alpha_{t+1}} \mathbf{x}_{t+1} - \alpha_t \left(\frac{\sigma_{t+1}}{\alpha_{t+1}} - \frac{\sigma_t}{\alpha_t} \right) \hat{\boldsymbol{\epsilon}}_{\theta}(\mathbf{x}(\lambda_{t+1}), \lambda_{t+1}) \\ &= \frac{\alpha_t}{\alpha_{t+1}} \mathbf{x}_{t+1} - \left(\alpha_t \frac{\sigma_{t+1}}{\alpha_{t+1}} - \sigma_t \right) \hat{\boldsymbol{\epsilon}}_{\theta}(\mathbf{x}(\lambda_{t+1}), \lambda_{t+1}) \\ &= \frac{\alpha_t}{\alpha_{t+1}} \mathbf{x}_{t+1} - \sigma_t (e^{h_t} - 1) \hat{\boldsymbol{\epsilon}}_{\theta}(\mathbf{x}(\lambda_{t+1}), \lambda_{t+1}) \end{aligned}$$

Recall: DDIM **Deterministic** Reverse Process

$$\begin{aligned} \mathbf{x}_t &= \sqrt{\bar{\alpha}_t} \left(\frac{1}{\sqrt{\bar{\alpha}_{t+1}}} \left(\mathbf{x}_{t+1} - \sqrt{1 - \bar{\alpha}_{t+1}} \hat{\boldsymbol{\epsilon}}_{\theta}(\mathbf{x}_{t+1}, t+1) \right) \right) + \sqrt{1 - \bar{\alpha}_t} \hat{\boldsymbol{\epsilon}}_{\theta}(\mathbf{x}_{t+1}, t+1) \\ &= \frac{\sqrt{\bar{\alpha}_t}}{\sqrt{\bar{\alpha}_{t+1}}} \mathbf{x}_{t+1} - \left(\frac{\sqrt{\bar{\alpha}_t}}{\sqrt{\bar{\alpha}_{t+1}}} \sqrt{1 - \bar{\alpha}_{t+1}} - \sqrt{1 - \bar{\alpha}_t} \right) \hat{\boldsymbol{\epsilon}}_{\theta}(\mathbf{x}_{t+1}, t+1) \\ &= \frac{\alpha_t}{\alpha_{t+1}} \mathbf{x}_{t+1} + \left(\alpha_t \frac{\sigma_{t+1}}{\alpha_{t+1}} - \sigma_t \right) \hat{\boldsymbol{\epsilon}}_{\theta}(\mathbf{x}_{t+1}, t+1) \end{aligned}$$

$$\begin{aligned} \alpha_t &= \sqrt{\bar{\alpha}_t} \\ \sigma_t &= \sqrt{1 - \bar{\alpha}_t} \end{aligned}$$

The first-order approximation is identical to DDIM!

DPM-Solver-2

First-Order Approximation:

$$\mathbf{x}_t = \frac{\alpha_t}{\alpha_{t+1}} \mathbf{x}_{t+1} - \sigma_t (e^{h_t} - 1) \hat{\boldsymbol{\epsilon}}_{\theta}(\mathbf{x}(t+1), t+1)$$

How about the **second-order** approximation?

DPM-Solver-2

1. **Split** the λ interval $(\lambda_t, \lambda_{t+1}]$ in half:

$$\bar{\lambda} = \frac{\lambda_t + \lambda_{t+1}}{2}$$

2. Perform the **first-order approximation** for the first half:

$$\mathbf{u} \leftarrow \frac{\alpha_t}{\alpha_{t+1}} \mathbf{x}_{t+1} - \sigma_{t\lambda}(\tilde{\lambda}) \left(e^{\frac{h_t}{2}} - 1 \right) \hat{\boldsymbol{\epsilon}}_{\theta}(\mathbf{x}(t+1), t+1)$$

3. Use the approximation to **update** $\hat{\boldsymbol{\epsilon}}_{\theta}$ and compute the final $\mathbf{x}_t$:

$$\mathbf{x}_t \approx \frac{\alpha_t}{\alpha_{t+1}} \mathbf{x}_{t+1} - \sigma_t(e^{h_t} - 1) \hat{\boldsymbol{\epsilon}}_{\theta}(\mathbf{u}, \tilde{\lambda})$$

How about the third-order approximation?

DPM-Solver- ***n***

Higher-Order Approximation

Taylor expansion of $\hat{\mathbf{e}}_{\theta}(x(\lambda), \lambda)$:

$$\hat{\mathbf{e}}_{\theta}(x(\lambda), \lambda) = \sum_{k=0}^{n-1} \frac{(\lambda - \lambda_{t+1})^k}{k!} \hat{\mathbf{e}}_{\theta}^{(k)}(x(\lambda_{t+1}), \lambda_{t+1}) + \eta$$

DPM-Solver-***n***

Plug it into the discrete setup formulation:

$$\begin{aligned}\mathbf{x}_t &= \frac{\alpha_t}{\alpha_{t+1}} \mathbf{x}_{t+1} - \alpha_t \int_{\lambda_{t+1}}^{\lambda_t} e^{-\lambda} \hat{\boldsymbol{\epsilon}}_{\theta}(\mathbf{x}(\lambda), \lambda) d\lambda \\ &= \frac{\alpha_t}{\alpha_{t+1}} \mathbf{x}_{t+1} - \alpha_t \int_{\lambda_{t+1}}^{\lambda_t} e^{-\lambda} \left(\sum_{k=0}^{n-1} \frac{(\lambda - \lambda_{t+1})^k}{k!} \hat{\boldsymbol{\epsilon}}_{\theta}^{(k)}(\mathbf{x}(\lambda_{t+1}), \lambda_{t+1}) + \eta \right) d\lambda \\ &= \frac{\alpha_t}{\alpha_{t+1}} \mathbf{x}_{t+1} - \alpha_t \sum_{k=0}^{n-1} \hat{\boldsymbol{\epsilon}}_{\theta}^{(n)}(\mathbf{x}(\lambda_{t+1}), \lambda_{t+1}) \int_{\lambda_{t+1}}^{\lambda_t} e^{-\lambda} \frac{(\lambda - \lambda_{t+1})^k}{k!} d\lambda + \eta\end{aligned}$$

Can be computed analytically

DPM-Solver-***n***

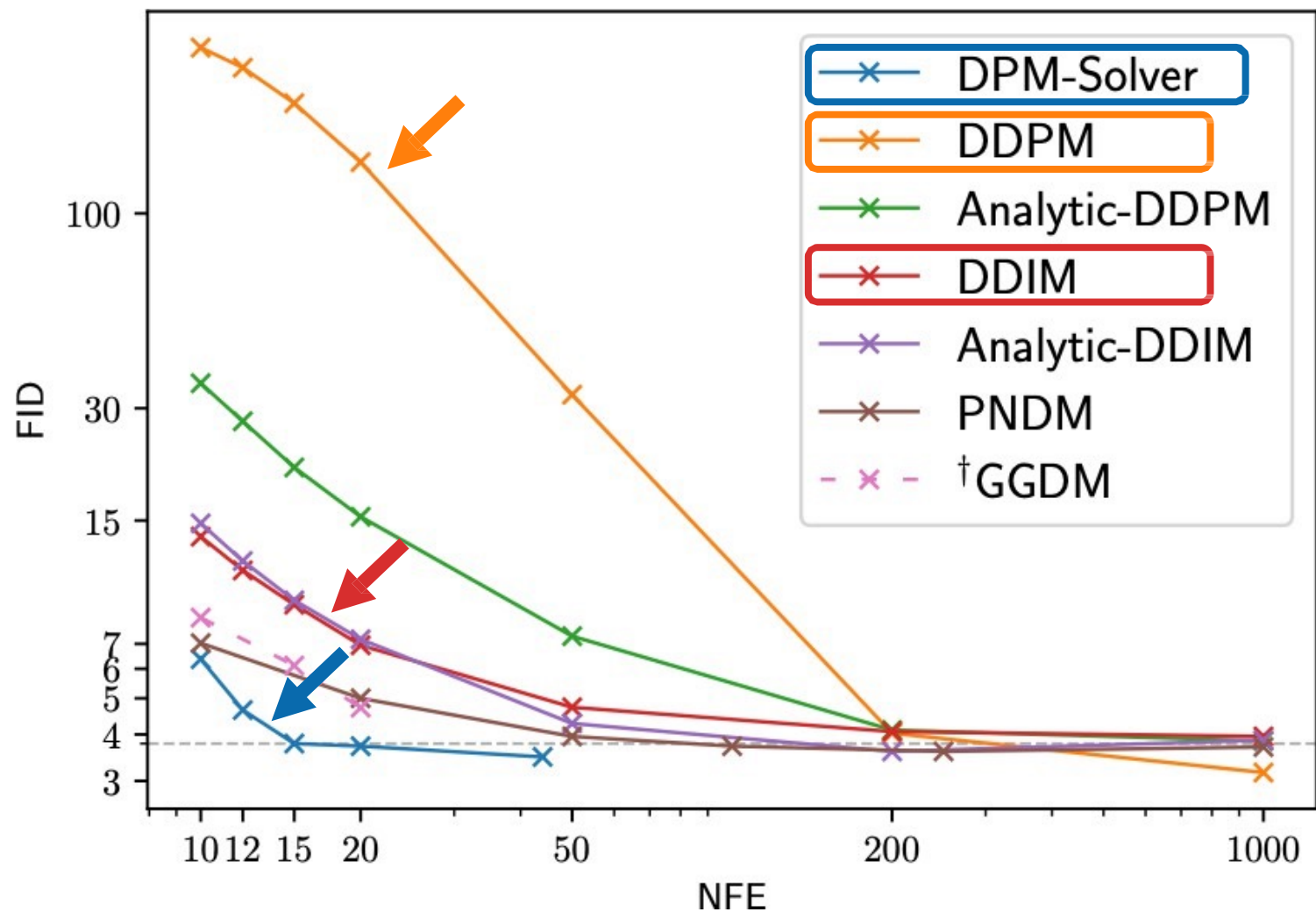
$$\boldsymbol{x}_t = \frac{\alpha_t}{\alpha_{t+1}} \boldsymbol{x}_{t+1} - \alpha_t \sum_{k=0}^{n-1} \hat{\boldsymbol{\varepsilon}}_{\theta}^{(n)}(x(\lambda_{t+1}), \lambda_{t+1}) h_t^{k+1} \varphi_{k+1}(h_t) + \eta$$

$$\varphi_1(h_t) = \frac{e^{h_t} - 1}{h_t}$$

$$\varphi_2(h_t) = \frac{e^{h_t} - h_t - 1}{h_t^2}$$

$$\varphi_3(h_t) = \frac{e^{h_t} - \frac{1}{2}h_t^2 - h_t - 1}{h_t^3}$$

DPM-Solver



複習思路：Discrete diffusion -> Continuous SDE -> ODE -> Taylor expansion for Higher-Order Approximation

DPM-Solver

與 DDIM 相似，不需要重新訓練 noise prediction network

假設從 $\mathbf{x}_T \rightarrow \mathbf{x}_0$ 需要 20 個 time steps，DDIM 的 (NFE) Number of Function Evaluations 就是 20，而 DPM-Solver-3 的 NFE 則是 $20 \times 3 = 60$

雖然達到同等生成品質的 NFE 減少了，但需要計算多個 φ 函數，且涉及指數運算和除法，所以 runtime 未必比簡單方法如 DDIM 快很多

對 noise prediction network 準確性要求高，從公式可見，依賴 $\hat{\epsilon}_{\theta}^{(n)}$ 這種 higher-order derivatives approximation（注意不用真的對 noise prediction network 算 gradient），如果 network prediction 不夠準確，誤差會被放大，且在 network 訓練不充分時效果可能不如預期

Further Readings

- Lu et al., DPM-Solver++: Fast Solver for Guided Sampling of Diffusion Probabilistic Models, 2023.
- Zheng et al., DPM-Solver-v3: Improved Diffusion ODE Solver with Empirical Model Statistics, NeurIPS 2023.
- Karras et al., Elucidating the Design Space of Diffusion-Based Generative Models, NeurIPS 2022.